Event ends in 2 days 2 hours 55 minutes.

Business logic implementation transforms your API specifications into robust, maintainable service layer architecture. This section creates detailed specifications for domain modeling, business rules, and clean architecture patterns.

## Exercise 2.1: Service Layer Architecture

Create comprehensive service layer architecture specifications:

```
Design detailed service layer architecture for our chatbot application:

**Use the Web Search MCP server to research current service architecture patterns, then create:**

**Service Layer Structure**:
```typescript
interface ServiceArchitecture {
  // Domain Services
  domainServices: {
    UserService: {
      responsibilities: ["User registration", "Authentication", "Profile management"];
      dependencies: ["UserRepository", "PasswordService", "TokenService"];
      methods: ["register", "authenticate", "updateProfile", "deactivateAccount"];
    };

    ConversationService: {
      responsibilities: ["Conversation management", "Message handling", "History retrieval"];
      dependencies: ["ConversationRepository", "MessageRepository", "AIService"];
      methods: ["createConversation", "sendMessage", "getHistory", "deleteConversation"];
    };

    AIService: {
      responsibilities: ["AI model integration", "Response generation", "Context management"];
      dependencies: ["BedrockClient", "ContextRepository", "TokenCounter"];
      methods: ["generateResponse", "processMessage", "manageContext"];
    };
  };

  // Application Services
  applicationServices: {
    ChatOrchestrationService: {
      purpose: "Orchestrate complete chat workflows";
      dependencies: ["ConversationService", "AIService", "NotificationService"];
      workflows: ["handleUserMessage", "processAIResponse", "updateConversation"];
    };

    AuthenticationService: {
      purpose: "Handle authentication workflows";
      dependencies: ["UserService", "TokenService", "AuditService"];
      workflows: ["loginWorkflow", "registrationWorkflow", "tokenRefreshWorkflow"];
    };
  };
}
```

**Domain Model Implementation**:
```typescript
// User Domain Model
interface UserDomain {
  entity: {
    id: UserId;
    email: Email;
    profile: UserProfile;
    preferences: UserPreferences;
    createdAt: Date;
    lastLoginAt: Date | null;
  };

  valueObjects: {
    Email: "Validated email with format checking";
    Password: "Encrypted password with strength validation";
    UserProfile: "Name, avatar, and personal information";
    UserPreferences: "Theme, language, notification settings";
  };

  businessRules: [
    "Email must be unique across all users",
    "Password must meet security requirements",
    "User profile must be complete for full access",
    "Inactive users are automatically archived after 2 years"
  ];
}

// Conversation Domain Model
interface ConversationDomain {
  entity: {
    id: ConversationId;
    userId: UserId;
    title: string;
    messages: Message[];
    metadata: ConversationMetadata;
    createdAt: Date;
    updatedAt: Date;
  };

  valueObjects: {
    Message: "Content, role, timestamp, and metadata";
    ConversationMetadata: "Token usage, model info, performance metrics";
    ConversationTitle: "Auto-generated or user-defined title";
  };

  businessRules: [
    "Conversations must have at least one message",
    "Message content cannot be empty",
    "System messages are automatically generated",
    "Conversations are soft-deleted for data retention"
  ];
}
```

Research current domain-driven design patterns and service architecture best practices.
```

### Expected Outcome

- Comprehensive service layer architecture with clear responsibilities
- Domain models with proper entity and value object separation
- Business rules clearly defined and enforced
- Clean separation between domain and application services
- Dependency injection patterns for testability and maintainability

## Exercise 2.2: Business Rules and Validation

Implement comprehensive business rules and validation logic:

```
Create detailed business rules and validation specifications:

**Validation Framework**:
```typescript
interface ValidationFramework {
  // Input Validation
  inputValidation: {
    userRegistration: {
      email: "Valid email format, not already registered";
      password: "12+ chars, mixed case, numbers, special chars";
      name: "Non-empty, reasonable length, no special chars";
    };

    messageValidation: {
      content: "1-4000 characters, no malicious content";
      conversationId: "Valid UUID, user has access";
      rate: "Respect rate limiting rules";
    };
  };

  // Business Rule Validation
  businessRules: {
    userManagement: [
      "Users can only access their own conversations",
      "Deleted users retain data for 30 days",
      "Premium users have higher rate limits"
    ];

    conversationManagement: [
      "Conversations auto-archive after 90 days inactive",
      "Message history limited to last 1000 messages",
      "AI responses must include safety filtering"
    ];
  };
}
```

**Error Handling and Recovery**:
```typescript
interface ErrorHandlingStrategy {
  // Error Categories
  errorCategories: {
    ValidationError: "Input validation failures";
    BusinessRuleError: "Business logic violations";
    ExternalServiceError: "Third-party service failures";
    SystemError: "Internal system failures";
  };

  // Recovery Strategies
  recoveryStrategies: {
    retryableErrors: "Exponential backoff with jitter";
    circuitBreaker: "Fail fast for degraded services";
    fallbackResponses: "Cached or default responses";
    gracefulDegradation: "Reduced functionality mode";
  };
}
```

Research current validation patterns and error handling strategies.
```

### Expected Outcome

- Comprehensive validation framework for all inputs and business rules
- Clear error categorization and handling strategies
- Recovery mechanisms for different types of failures
- Consistent validation patterns across all services
- Proper error propagation and logging

## Exercise 2.3: Data Access and Repository Patterns

Design data access layer with repository patterns:

```
Create detailed data access specifications:

**Repository Pattern Implementation**:
```typescript
interface RepositoryPatterns {
  // Base Repository Interface
  baseRepository: {
    methods: ["findById", "findAll", "create", "update", "delete"];
    patterns: ["Generic repository with type safety"];
    caching: "Implement caching at repository level";
  };

  // Specific Repositories
  userRepository: {
    methods: ["findByEmail", "findActiveUsers", "updateLastLogin"];
    queries: "Optimized queries for user operations";
    indexing: "Proper indexing for email and ID lookups";
  };

  conversationRepository: {
    methods: ["findByUserId", "findWithMessages", "archiveOldConversations"];
    pagination: "Efficient pagination for large datasets";
    relationships: "Proper handling of conversation-message relationships";
  };
}
```

Research current repository pattern implementations and data access best practices.
```

### Expected Outcome

- Repository pattern implementation with proper abstraction
- Optimized data access patterns for performance
- Caching strategies at the data access layer
- Proper handling of relationships and transactions
- Type-safe data access with comprehensive error handling

## Key Deliverables

After completing the Business Logic exercises, you should have:

1. **Service Layer Architecture**: Clean separation of domain and application services
2. **Domain Models**: Proper entity and value object modeling
3. **Business Rules**: Comprehensive validation and rule enforcement
4. **Data Access Patterns**: Repository pattern with caching and optimization
5. **Error Handling**: Robust error handling and recovery strategies

## Next Steps

With business logic architecture established, you're ready to implement comprehensive data management strategies that efficiently store, retrieve, and manage your chatbot application's data.